Experiment NO 6

Name: Parth Vijay Gulhane

Roll No: 23CO043

Class: TE Comp A

Aim

Implement any one of the following Expert System I. Information management II. Hospitals and medical facilities III. Help desks management IV. Employee performance evaluation V. Stock market trading VI. Airline scheduling and cargo schedules

Objectives

- To understand the concept of Expert Systems in Artificial Intelligence.
- To design a system using knowledge base and inference engine.
- To simulate medical diagnosis based on symptoms.
- To apply rule-based reasoning techniques.
- To reduce human effort in preliminary diagnosis.
- To demonstrate real-world applications of AI in healthcare systems.

Theory

An Expert System is a branch of Artificial Intelligence that mimics the decision-making ability of a human expert. It uses knowledge and inference techniques to solve complex problems.

This experiment is based on the following SPPU AI syllabus topics:

- Expert Systems
- Knowledge Representation
- Inference Mechanism
- Rule-Based Systems
- Artificial Intelligence in Healthcare

1. Components of Expert System

Component	Description
Knowledge Base	Stores facts and rules (diseases & symptoms)
Inference Engine	Applies logic to derive conclusions
User Interface	Interacts with the user
Explanation System	Provides reasoning (optional)

2. Knowledge Representation

Knowledge is represented using rules and facts. **Example**

Rule:

IF patient has fever AND cough AND tiredness
THEN possible disease = COVID-19

3. Inference Engine

The inference engine processes user input and matches it with stored rules.

Types of Reasoning:

Type	Description
Forward Chaining	Starts from symptoms → finds disease
Backward Chaining	Starts from disease → checks symptoms

In this experiment, Forward Chaining is used.

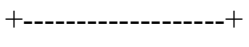
4. Working of the System

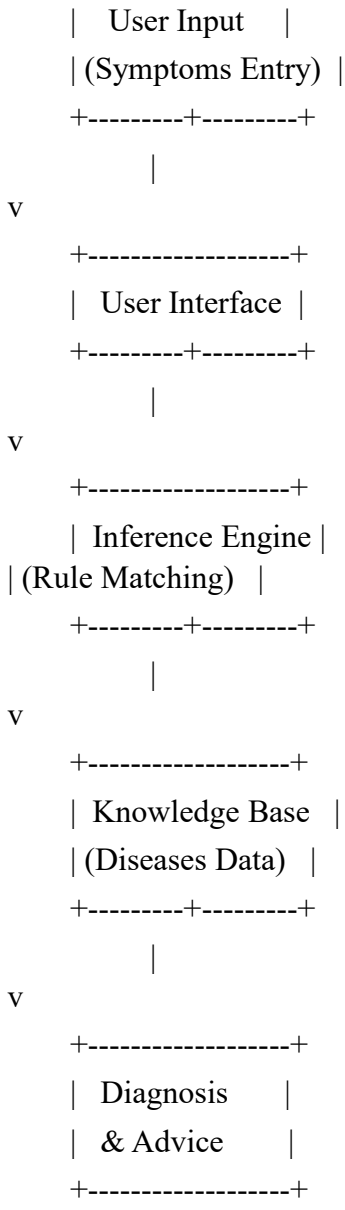
1. User enters symptoms (Yes/No)
2. System collects all symptoms
3. Inference engine compares symptoms with knowledge base
4. Matching percentage is calculated
5. Most probable disease is displayed

5. Knowledge Base Table

Disease	Symptoms
Common Cold	cough, sneezing, sore throat
Influenza (Flu)	fever, chills, body ache
COVID-19	fever, dry cough, breathing difficulty
Food Poisoning	vomiting, diarrhea, nausea
Dengue	high fever, joint pain, rash

6. System Architecture Diagram





7. Applications

Domain	Application
Hospitals	Patient diagnosis
Domain	**Application**
Clinics	Symptom checking
Telemedicine	Online consultation

Healthcare Apps AI health assistants

Algorithm

1. Start
2. Initialize knowledge base
3. Accept symptoms from user
4. Compare symptoms with diseases
5. Calculate matching percentage
6. Select highest match
7. Display diagnosis and advice
8. Stop

Conclusion

The expert system for hospitals and medical facilities successfully demonstrates the application of Artificial Intelligence concepts such as knowledge representation, inference engine, and rulebased systems. It efficiently analyzes symptoms and suggests possible diseases with a matching percentage.

This system shows how AI can assist in healthcare by providing quick preliminary diagnosis and decision support. Although it cannot replace medical professionals, it serves as a valuable tool for improving efficiency and accessibility in healthcare systems.

CODE

```
#include <iostream>
#include <vector>
#include <map>
#include <set>
#include <algorithm>
#include <thread>
#include <chrono>
using namespace std;
// ANSI Colors
class Style {
public:
    static constexpr const char* HEADER = "\033[95m";
    static constexpr const char* BLUE   = "\033[94m";
    static constexpr const char* CYAN   = "\033[96m";
    static constexpr const char* GREEN  = "\033[92m";
    static constexpr const char* YELLOW = "\033[93m";
    static constexpr const char* RED    = "\033[91m";
    static constexpr const char* BOLD   = "\033[1m";
    static constexpr const char* RESET  = "\033[0m";
};
class MedicalExpertSystem {
private:
    map<string, vector<string>> knowledge_base;
    vector<string> patient_symptoms;
public:
```

```
    MedicalExpertSystem() {
        // Knowledge Base
        knowledge_base = {
            {"Common Cold", {"cough", "runny_nose", "sneezing", "sore_throat",
"mild_fever"}},
            {"Influenza (Flu)", {"high_fever", "chills", "body_ache", "headache",
"fatigue"}},
            {"COVID-19", {"fever", "dry_cough", "loss_of_taste_or_smell",
"difficulty_breathing", "tiredness"}},
            {"Food Poisoning", {"nausea", "vomiting", "diarrhea", "abdominal_pain",
"mild_fever"}},
            {"Dengue", {"high_fever", "severe_headache", "pain_behind_eyes",
"joint_pain", "skin_rash"}}
        };
    }
    void print_header() {
        cout << Style::CYAN << Style::BOLD;
        cout <<
"╔═══════════════════════════════════════════════════════════════════════
═══════════════╗\n";
        cout << "║          🏥 MEDICAL DIAGNOSIS EXPERT SYSTEM          ║\n";
        cout <<
"╚═══════════════════════════════════════════════════════════════════════
═══════════════╝\n";
        cout << Style::RESET;
        cout << Style::YELLOW << "Instructions:" << Style::RESET
             << " Please answer the following symptom checks.\n";
        cout << string(58, '-') << "\n\n";
```

```
}
vector<string> get_all_unique_symptoms() {
    set<string> all;
    for (auto &entry : knowledge_base) {
        for (auto &sym : entry.second) {
            all.insert(sym);
        }
    }
    return vector<string>(all.begin(), all.end());
}
void ask_questions() {
    print_header();
    vector<string> symptoms = get_all_unique_symptoms();
    for (auto &symptom : symptoms) {
        string formatted = symptom;
        replace(formatted.begin(), formatted.end(), '_', ' ');
        while (true) {
            cout << Style::BLUE << "❯ " << Style::RESET
                 << "Do you have " << Style::BOLD << formatted
                 << Style::RESET << "? (y/n): ";
            string response;
            cin >> response;
            transform(response.begin(), response.end(), response.begin(), ::tolower);
            if (response == "y" || response == "yes") {
                patient_symptoms.push_back(symptom);
                break;
```

```
            }
            else if (response == "n" || response == "no") {
                break;
            }
            else {
                cout << Style::RED
                     << "[!] Invalid input. Please type 'y' or 'n'.\n"
                     << Style::RESET;
            }
        }
    }
}
void simulate_processing() {
    cout << "\n" << Style::YELLOW << "Processing Data" << Style::RESET;
    for (int i = 0; i < 3; i++) {
        this_thread::sleep_for(chrono::milliseconds(400));
        cout << ".";
    }
    cout << "\n\n";
}
void diagnose() {
    simulate_processing();
    map<string, double> diagnosis_scores;
    for (auto &entry : knowledge_base) {
        string disease = entry.first;
        vector<string> symptoms = entry.second;
```

```
        int matches = 0;
        for (auto &ps : patient_symptoms) {
            if (find(symptoms.begin(), symptoms.end(), ps) != symptoms.end()) {
                matches++;
            }
        }
        double score = ((double)matches / symptoms.size()) * 100;
        diagnosis_scores[disease] = score;
    }
    // Find max score
    string likely_disease;
    double max_score = -1;
    for (auto &entry : diagnosis_scores) {
        if (entry.second > max_score) {
            max_score = entry.second;
            likely_disease = entry.first;
        }
    }
    display_results(likely_disease, max_score);
}
void display_results(string disease, double score) {
    cout << Style::GREEN << Style::BOLD;
    cout <<
"╔══════════════════════════════════════════════════════════════╗\n";
    cout << "║              📋 DIAGNOSIS REPORT              ║\n";
```

```
        cout <<
"╚═══════════════════════════════════════════════════════════════
═══════════════╝\n";
        cout << Style::RESET;
        if (score == 0) {
            cout << "\nCondition: No matching disease found.\n";
            cout << "Match: 0.0%\n";
            cout << "Advice: Consult a doctor.\n";
        }
        else if (score >= 60) {
            cout << "\nCondition: " << Style::RED << disease << Style::RESET <<
"\n";
            cout << "Match: " << Style::RED << score << "%" << Style::RESET <<
"\n";
            cout << "Advice: Seek medical attention.\n";
        }
        else {
            cout << "\nCondition: " << Style::CYAN << disease << Style::RESET <<
"\n";
            cout << "Match: " << Style::CYAN << score << "%" << Style::RESET <<
"\n";
            cout << "Advice: Rest and monitor symptoms.\n";
        }
        cout << "\n" << string(58, '-') << "\n";
    }
};

int main() {
```

```
    try {
        MedicalExpertSystem system;
        system.ask_questions();
        system.diagnose();
    }
    catch (...) {
        cout << Style::RED << "[!] System terminated by user." << Style::RESET << endl;
    }
    return 0;
}
```

OUTPUT